

Technical Assessment on Neural Reservoir Surrogate: X-MeshGraphNet on Norne

Setup & Issues Encountered

Hardware: GTX 1080 (8 GB VRAM), WSL2 Ubuntu 24.04, CUDA 13.0. Dataset placed on native ext4 (`~/completed/`), which is critical on WSL2: paths through `/mnt/d/` reduce GPU utilisation to $\sim 0\%$ via the 9p filesystem bridge. WSL2 itself was installed to the D: drive to accommodate the ~ 140 GB preprocessed dataset. The following bugs were encountered and fixed during the run:

1. **MLflow FileStore exception.** MLflow $\geq 2.x$ deprecates filesystem tracking by default. Fix: prepend `MLFLOW_ALLOW_FILESTORE=true` to all train and inference commands.
2. **GPU contention crash at epoch 70.** Training was interrupted when a GPU-intensive application was accidentally launched on the host. The GTX 1080 has only 8 GB VRAM and training already consumed ~ 7 GB, leaving no headroom. The process was killed by the OS. Training resumed cleanly from the auto-saved checkpoint using `resume: true` in the config with no data loss. MLflow logged this as two separate runs (epochs 1–70 and 71–120), both shown in the loss curve screenshots below.
3. **inspect_graphs.py line 34 typo.** tuple type not printing properly. Fix: wrap tuple object in `str()`.
4. **inference.py shape mismatch in Patch 7 reorder block (lines 578–585).** `reorder_to_natural` was called on per-partition arrays ($\sim 22\,843$ nodes each) instead of the concatenated full-case array (44 431 nodes). Fix: call `np.concatenate(vec_list, axis=0)` before reordering.
5. **final_timestep_matrix.py hardcoded paths.** Four path constants pointed to the exercise author’s machine. Updated to local paths; also corrected `{case}.UNRST` to `{case}_PRED.UNRST` on line 137.
6. **Missing resdata package.** Fix: `pip install resdata --break-system-packages`.

Preprocessing & Data Split

60 simulation cases processed (3 720 graphs total, 62 timestep graphs per case), 2 METIS partitions per graph, halo size 3. Each graph: 44 431 active nodes, $\sim 135\,000$ edges, 13 node features, 1 edge feature, 2 target variables (PRESSURE, SWAT).

Split (`random_seed: 42`): 48 train / 6 val / 6 test.

Test cases (held out, not seen during training): NORNE_002, NORNE_008, NORNE_016, NORNE_018, NORNE_041, NORNE_048

Model Configuration & Rationale

Architecture: X-MeshGraphNet (encode-process-decode GNN, NVIDIA PhysicsNeMo).

```
model:      num_message_passing_layers: 3    hidden_dim: 64    activation: silu
training:   start_lr: 1e-4    end_lr: 1e-6    weight_decay: 1e-3
            early_stopping:  patience: 20    (stops after 20 epochs without improvement)
target_vars: loss_functions: [L2, L1]    weights: [1.0, 1.0]
```

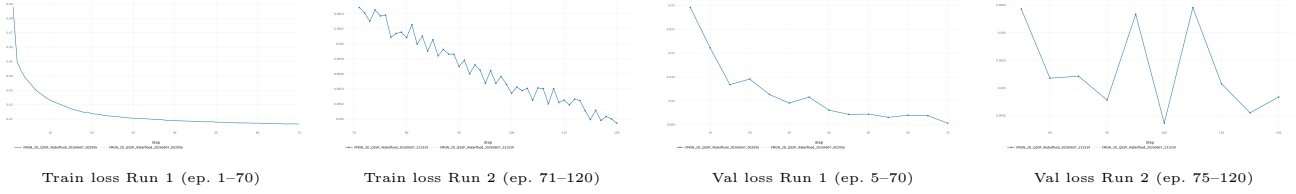
This is a deliberate **baseline run**. With 44 431 nodes and only 8 GB VRAM, `hidden_dim: 64` and 3 layers already consume ~ 7 GB. Running the default config first establishes a performance floor and validates the end-to-end pipeline before committing to longer, larger experiments.

Loss function choice: L2 for PRESSURE penalises large errors quadratically, appropriate since pressure errors scale with physical magnitude. L1 for SWAT is more robust to the sparse structure of saturation front errors: many near-zero cells, few large errors at flood fronts. Equal weights `[1.0, 1.0]` kept symmetric in the absence of domain evidence for a different trade-off.

Training Run

Hardware: GTX 1080, $\sim 77\%$ GPU utilisation, ~ 15 – 17 min/epoch. **Total time:** ~ 33 hours across two sessions (session 1: epochs 1–70, 20.3 hrs; session 2: epochs 71–120, 12.5 hrs). **Best checkpoint:** epoch 100. Early stopping triggered at epoch 120 (20 epochs without improvement).

Epoch	Val loss (norm.)	PRES MAE (bar)	PRES RMSE (bar)	SWAT RMSE
5	0.02957	3.53	6.62	0.01225
15	0.01339	2.15	3.84	0.01008
40	0.00805	1.20	2.27	0.00706
70	0.00533	0.92	1.70	0.00549
100	0.00437	0.84	1.51	0.00442
120	0.00484		(early stop triggered)	



Run 1 train loss shows rapid descent from 0.088 at epoch 1 to ~ 0.007 by epoch 70 before the GPU crash. Run 2 train loss resumes smoothly from ~ 0.0065 and continues downward to ~ 0.0049 at epoch 120, confirming a clean resume with no loss spike. Run 2 val loss oscillates rather than descending monotonically, indicating sensitivity to which validation graphs are sampled each check, but the overall trend is downward. It bottoms at 0.00437 at epoch 100 before rising to 0.00484 at epoch 120, confirming the early stopping decision.

Test-Set Metrics

Autoregressive rollout: first 3 timesteps use true features as input; the remaining 59 timesteps use model predictions fed back as input.

	PRESSURE		SWAT	
	MAE	RMSE	MAE	RMSE
Normalised	0.094904	0.189007	0.008898	0.036586
Physical	4.6304 bar	9.2217 bar	0.003174	0.013051

PRESSURE MAE (4.63 bar): Average per-cell pressure error across all nodes and autoregressive steps. At a mean reservoir pressure of ~ 278 bar this is a relative error (MAE/mean) of $\sim 1.7\%$, reasonable for a baseline model.

PRESSURE RMSE (9.22 bar): Higher than MAE because timestep 13 alone reaches 12.33 bar, pulling the aggregate up. RMSE penalises large individual errors more heavily than MAE, so the gap between them signals that errors are concentrated in specific timesteps rather than spread evenly.

SWAT MAE (0.003174) and RMSE (0.01305): MAE appears small because most cells are fully water-saturated and already near the correct value, contributing little error. RMSE is proportionally much larger because errors at the remaining cells which just touch the front of water are locally large, and RMSE amplifies those few large values more than MAE does.

Per-case PRESSURE RMSE: NORNE_008: 8.94 bar (best); NORNE_048: 9.02; NORNE_018: 9.10; NORNE_041: 9.22; NORNE_016: 9.33; NORNE_002: 9.70 bar (worst). The spread of only 0.76 bar across 6 cases with different fault multiplier configurations suggests the model is not strongly conditioning on fault configuration.

Error stratified by rollout timestep (selected):

Timestep	PRES RMSE (bar)	SWAT RMSE	Timestep	PRES RMSE (bar)	SWAT RMSE
3	0.63	0.00059	33	7.10	0.01627
8	7.13	0.01175	38	5.12	0.00757
13	12.33	0.03629	53	10.47	0.01170
18	7.77	0.01972	63	9.57	0.00506
23	3.51	0.00454			

Timestep 3 error is very low (0.63 bar) as it is the first predicted step immediately after the warm-up phase where true features were used as input, so compounding has not yet begun. Error rises sharply at timestep 8 (7.13 bar) as autoregressive compounding takes hold: each prediction is now fed back as the next input, so any error in the previous step is inherited and built upon. The spike at timestep 13 (12.33 bar PRESSURE, 0.036 SWAT) is non-monotonic and discussed in detail in the next section. The partial recovery at timestep 23 (3.51 bar) and subsequent irregular pattern reflect the changing physical regime rather than model self-correction, as error in an autoregressive model can only accumulate, never reduce.

Failure Mode Analysis

1. Error spike at timestep 13, followed by recovery

PRESSURE RMSE peaks at 12.33 bar at timestep 13, then drops sharply to 3.51 bar at timestep 23. Standard autoregressive drift predicts monotonically growing error; this spike-then-recovery pattern indicates a physical amplification mechanism rather than simple error accumulation.

Hypothesis: A compounding factor is the model's **limited receptive field**: 3 message-passing layers propagate information at most 3 hops per forward pass. Norne's longest flow paths across NNC fault edges span substantially more than 3 cells, so pressure signals cannot cross major fault boundaries in a single forward pass.

2. Narrow inter-case PRESSURE RMSE spread (8.94–9.70 bar)

The per-case spread is narrow despite test cases spanning the full fault-multiplier uncertainty range. This suggests the model predicts a near-constant mean-field pressure evolution rather than a fault-conditioned one.

Hypothesis: `hidden_dim: 64` bottlenecks the encoder’s ability to distinguish cases that differ only in NNC transmissibility weights. The fault multiplier signal is present in edge features but may not survive 62 autoregressive steps at this capacity.

3. Spatial error structure



TRUE / PRED / DIFF at final timestep. Rows: BEST, MEDIAN, WORST cases. Top-down view, mean over Z.

PRESSURE diff panels (scale ± 10 bar) show a consistent warm bias ($\sim +5$ bar over-prediction) in the southern segment and a cool bias in the northern stepped segment (top right), sharpest at the inter-segment boundary which likely arises from a fault. This is consistent with the receptive-field limitation: 3 message-passing layers cannot propagate pressure signals across the major fault separating the two segments. SWAT truth panels show

the reservoir is almost entirely full of water at the final timestep (dark blue) as expected as water is continuously injected, with saturation confined to the north eastern segment. This nearly binary spatial field (as most of the space is full of water) explains why aggregate SWAT RMSE is low (0.013) while local errors are large in some places: the majority of cells contribute near-zero error, masking the model’s over-prediction of saturation at the top right segment.

What I Would Do Next

Next run: `hidden_dim: 128, num_message_passing_layers: 6, halo_size: 5`. This directly addresses the receptive-field limitation responsible for the timestep-13 spike and the inter-segment pressure bias by doubling model capacity and extending the effective neighbourhood to 6 hops.

Architecture improvements, each motivated by a specific failure mode:

- (1) *Physics-informed loss* (motivated by the timestep-13 spike): add a penalty to constrain autoregressive drift especially during the phase where pressure and SWAT start to amplify.
- (2) *Explicit MULTFLT conditioning* (motivated by the narrow inter-case spread): inject per-fault multiplier scalars as global context vectors so the model can condition on fault configuration beyond what NNC edge weights alone encode across 62 steps.
- (3) *Longer history window, prev.timesteps: 4* (motivated by mis-prediction and autoregressive drift): better captures trends as the flow of fluid is heavily time dependent and reduces sensitivity to the rapid saturation advance around timestep 13.

Bonus: FNO vs. X-MGN for Faulted Reservoirs

FNO operates in Fourier space on a *regular* grid, assuming the input and output fields live on a tensor-product lattice implying a clean regular geometry which isn’t the case in most real world reservoirs due to faults. Norne’s ~ 47 faults are encoded as NNCs: edges between grid cells that are not spatially adjacent in i, j, k coordinates. An NNC connecting $(i=10, j=20)$ to $(i=10, j=45)$ across a fault plane has no meaningful Fourier representation. An FNO applied to Norne must either ignore NNCs entirely (causing systematic pressure errors at every fault boundary) or encode them as auxiliary input on the regular grid (misrepresenting long-range topology as a local field). Either choice is architecturally incorrect.

X-MGN’s message-passing is topology-agnostic: NNC edges are graph edges with their own weights, treated identically to regular cell-neighbour edges. The graph structure here is not a convenience; it is a necessary architectural choice for faulted reservoir geometry.

Where FNO wins: unfaulted reservoirs with regular geometry and no NNCs. FNO’s complexity and global spectral receptive field outperform GNN layer-stacking for smooth, structured-grid problems.

Deliverables: (1) `best_checkpoints/checkpoint.0.100.pt` (2) This writeup (3) `conf/config.yaml` (4) 6 inference HDF5 files (`outputs/.../inference/`) (5) `final_timestep_matrix.png`

Bonus: FNO vs. X-MGN · Error stratified by timestep (Test-Set Metrics) · Architecture proposals (What I Would Do Next)